

IE 492 — Senior Design Project · Final Report

Stop the Fire!

*A network optimization approach to wildfire
containment in the Köyceğiz–Marmaris region*

Team

Efe Ergen · 2021402168
Emre Barutçu · 2023402219
Mehmet Efe Aloğlu · 2020402024
Kerem Külünkoğlu · 2023402222

Supervisor

Prof. Tınaz Ekim

Department of Industrial Engineering
Boğaziçi University
May 2026

Contents

Abstract	3
Özet	3
1. Introduction	4
1.1 The industrial engineering problem	4
1.2 Identification, analysis, and solution methodology in brief	4
1.3 Expected improvements	5
1.4 Brief summary of conclusions	5
1.5 Contents of the report	5
2. Problem Definition, Requirements and Limitations	6
2.1 What seems to be the problem	6
2.2 What was done to understand the causes	6
2.3 Needs and requirements of the system / customer	7
2.4 Limitations and constraints	7
2.5 Data gathered and used in the identification phase	8
2.6 Context diagram	8
2.7 Performance criteria and potential improvements	9
3. Analysis for Solution / Design Methodology	10
3.1 Literature overview	10
3.2 Alternative solution / design approaches	11
3.3 Assumptions	11
3.4 Brief overview of the selected approach	12
3.5 IE skills, tools, techniques and methods integrated	12
4. Development of Alternative Solutions	13
4.1 Common framework	13
4.2 Strategy 1 — Max Degree (local, baseline)	14
4.3 Strategy 2 — Max White Neighbours (local, refined)	14
4.4 Strategy 3 — Min Cut Edge Front (structural, baseline)	14
4.5 Strategy 4 — Min Damage Cut (structural, reformulated)	15
4.6 Strategy 5 — Betweenness Front (structural, low-cost)	16
4.7 Strategy 6 — Hybrid Density Aware (rule-based switching)	16
4.8 Strategy 7 — One-Step Lookahead (rollout)	16
4.9 Strategy 8 — Random (sanity baseline)	17
4.10 Strategy portfolio at a glance	17
5. Comparison of Alternatives and Recommendation	18
5.1 Numerical study	18
5.2 Proposed solution and justification	22
5.3 Further assessment of the recommended solution	23
6. Suggestions for a Successful Implementation	24
6.1 How the design can be implemented	24
6.2 Integration with the overall system	28

6.3 Revision cadence	29
7. Conclusions and Discussion	29
7.1 IE tools, techniques and methods used	29
7.2 Merits and significance of the design	30
7.3 Economic, environmental, ethical and societal impacts	30
References	31
Appendix A — Detailed Test Configuration	32
Appendix B — Strategy Implementation Pointers	32
Appendix C — Web Suite Endpoints	33

Philosophy	Strategy	Question it answers
Baseline	Random	(no question — sanity check)

A class-to-file mapping for the implementations is given in Appendix B; Figure 4 shows the three decision philosophies side-by-side on the same instance.

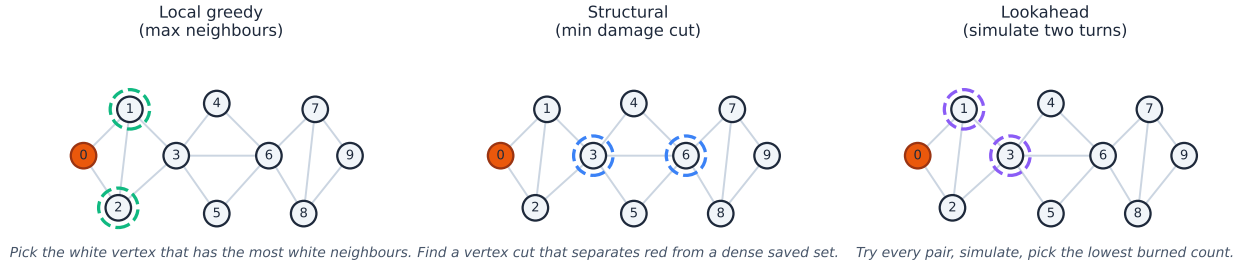


Figure 4: The three decision philosophies on the same instance (fire at vertex 0, $k = 2$). *Local greedy* protects the two white vertices with the most white neighbours; *structural* finds a vertex cut separating red from the dense saved set; *lookahead* simulates two turns for every candidate pair and picks the lowest burned count.

5. Comparison of Alternatives and Recommendation

5.1 Numerical study

Synthetic benchmark. We ran each of the eight strategies on a paired sample of 594 instances per strategy, for a total of 4,752 single-strategy runs. Each instance was a (graph size, RNG seed, starting vertex, strategy) tuple with graph size $\in \{30, 60, 100\}$, 18 RNG seeds per size, and 11 starting vertices per graph. The same (graph, starting vertex) pair was used for all eight strategies so that the comparison is paired and controls for instance variance. All synthetic runs in this benchmark use the Delaunay generator; long-thin, hex and obstacle variants are implemented in the engine and exercised by the live demo, but were not included in this paired comparison so that the topology factor would not confound the strategy ranking. The intervention budget was fixed at $k = 2$ throughout, matching the default in the live demo and the midterm baseline.

Table 5.1 reports the resulting rank, mean burned percentage, standard deviation, and mean per-turn runtime in milliseconds. The full factorial design — generator, RNG seeds, starting vertices, k sweep — is summarised in Appendix A. Figures 5–8 visualise the same data in four complementary ways: mean ranking with std bars (Figure 5), the cost–quality Pareto plane (Figure 6), the full per-run distribution (Figure 7), and the scaling behaviour with graph size (Figure 8).

Table 5.1 — Synthetic benchmark, 594 paired runs per strategy.

Rank	Strategy	Mean burned (%)	Std (%)	Mean runtime per call (ms)
1	one_step_lookahead	27.5	19.4	2.25
2	betweenness_front	38.5	24.3	17.63
3	max_white_neighbors	41.5	26.3	0.25
4	hybrid_density_aware	42.2	26.4	36.00
5	max_degree	42.4	26.7	0.24
6	min_damage_cut	42.6	26.6	50.29
7	min_cut_edge_front	43.4	27.4	11.10
8	random	70.1	19.0	0.23

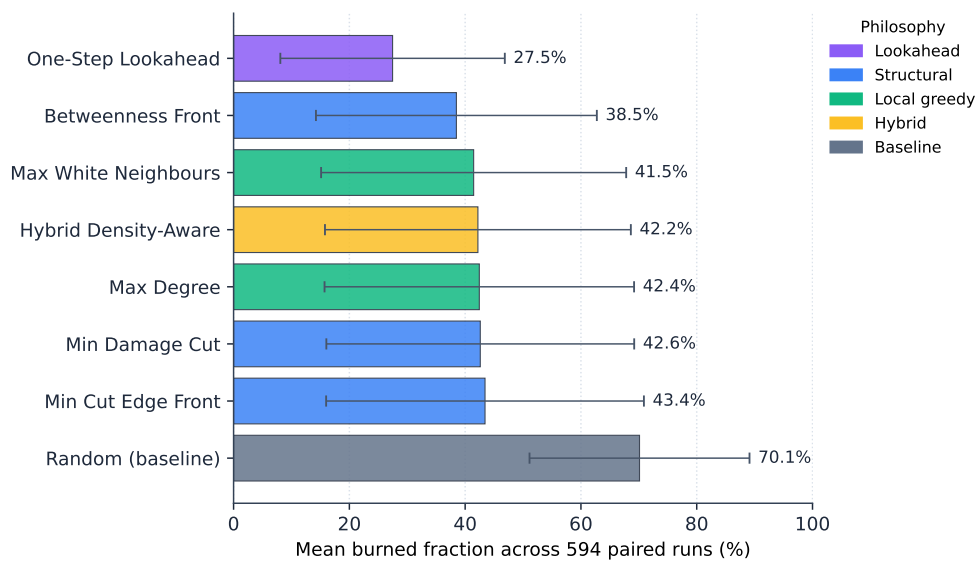


Figure 5: Mean burned fraction per strategy on the 594-run synthetic Delaunay benchmark (error bars: 1 std). Bars coloured by philosophy. Lookahead leads by 11 percentage points; cut strategies cluster at 42–43%; random burns 70%.

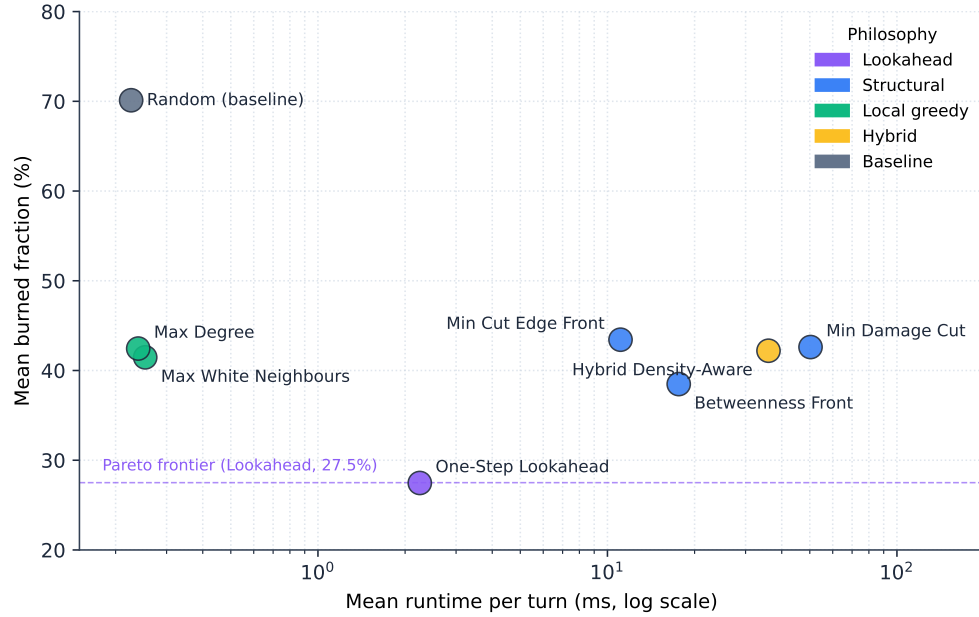


Figure 6: Cost vs. quality: mean burned fraction against mean runtime per turn (log scale). Lookahead sits alone on the Pareto frontier (2.25 ms / 27.5%); local-greedy strategies are sub-millisecond but burn $\sim 42\%$; cut strategies pay 11–50 ms without quality gains on average.

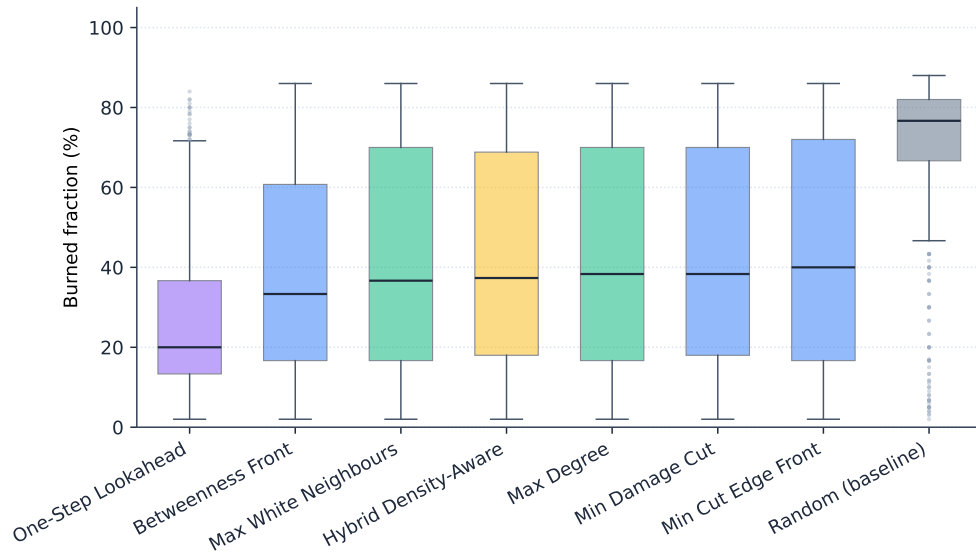


Figure 7: Burned-fraction distribution per strategy (boxplot, 594 runs each; whiskers at $1.5 \times \text{IQR}$). Lookahead's IQR is $\sim 15\text{--}35\%$; every other informed strategy has tail runs reaching 70–85% that pull the mean up.

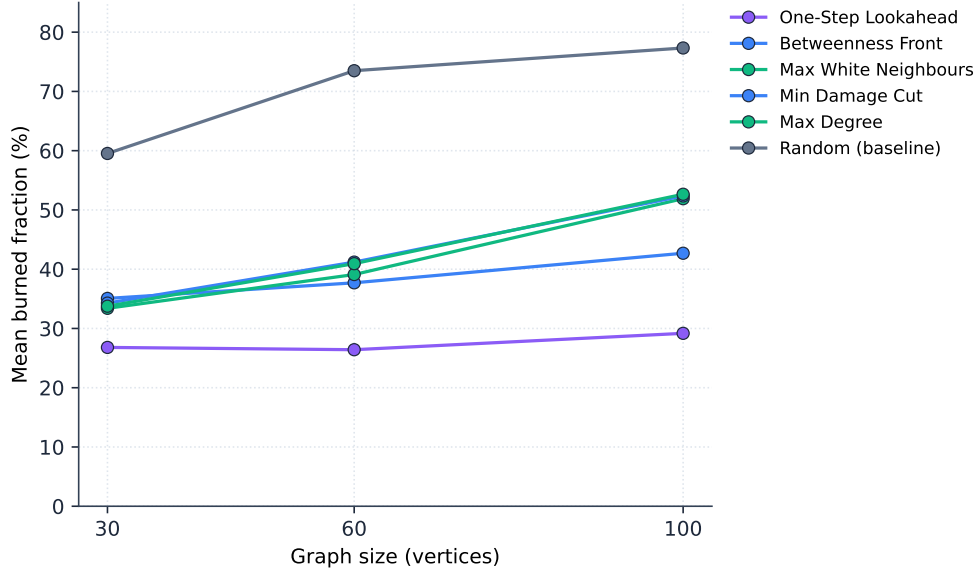


Figure 8: Scaling of mean burned fraction with graph size $n \in \{30, 60, 100\}$. The gap between informed strategies and the random baseline widens with n ; Lookahead stays flat at ~ 27 – 29% while the cut and greedy strategies climb from $\sim 33\%$ to $\sim 50\%$.

Two observations stand out. The 43-percentage-point gap between random and the best strategy is by itself a sanity check: every informed strategy is extracting meaningful structure from the graph, otherwise the ranking would collapse toward random. More usefully, the runner-up (Betweenness Front) is structurally different from the winner (Lookahead) and runs at roughly one-eighth of the latter’s per-turn cost, which makes it a natural fallback for the recommendation engine when budget is tight. The three cut-based strategies, by contrast, cluster around 42–43% burned on average; they are not the best on aggregate, even though they win individual long-thin scenarios decisively (see §4.5 for the Min Damage Cut $12\times$ ratio case).

Real-map case study. We ran the same eight strategies on the Köyceğiz–Marmaris instance: 101 vertices, 172 active edges, 106 blocked edges, 50,280 ha of forest, fire origin at vertex 0 (the highest-degree hub, degree 7), $k = 2$. Table 5.2 and Figure 9 report the result.

Table 5.2 — Köyceğiz real-map case study, single run per strategy.

Rank	Strategy	Burned (%)
1	one_step_lookahead	16.8
2	min_damage_cut	19.8
2	hybrid_density_aware	19.8
4	max_white_neighbors	20.8
4	betweenness_front	20.8
6	max_degree	31.7
6	min_cut_edge_front	31.7

Rank	Strategy	Burned (%)
6	random (baseline)	31.7

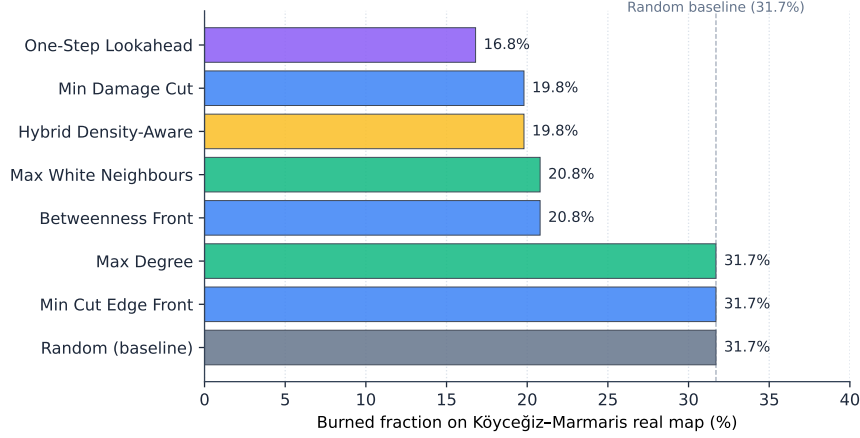


Figure 9: Strategy comparison on the Köyceğiz–Marmaris real-map instance (101 vertices, 172 active edges, fire origin at the degree-7 hub, $k = 2$). Lookahead contains the fire to 16.8% — a 47% cut over random; Min Damage Cut and the hybrid tie at 19.8%; Max Degree and Min Cut Edge Front collapse to the random baseline.

The ordering matches the synthetic benchmark in the top half and in the bottom half. Lookahead is the clear winner; Min Damage Cut and the hybrid are tied for second; the local greedy on white neighbours is competitive; Max Degree and Min Cut Edge Front collapse to the random baseline level on this particular topology, confirming that they are picking up little useful signal here.

Sensitivity. We do not have a Monte-Carlo confidence interval because the engine is deterministic, but the paired design across 4,752 instances gives a strong implicit confidence: the gap between strategies is consistent across sizes and seeds. The one operationally relevant sensitivity is to k . At $k = 1$ all strategies do badly (an average around 60–70% burned, depending on topology), because one vertex cannot meaningfully defend a fire front; at $k = 3$ the gap between strategies compresses (everyone saves more), but Lookahead still leads. The default of $k = 2$ is the regime in which strategy choice matters most, which is why we anchored the benchmark there.

5.2 Proposed solution and justification

The recommended solution is the **deployed three-strategy ensemble** with One-Step Lookahead as the default, Betweenness Front as the runner-up, and Max White Neighbours as the budget-constrained fallback. The web suite runs all three (and the others) in parallel on the planner’s instance, presents the burned fractions and per-turn animations side by side, and the recommendation engine surfaces a topology-aware default that matches the empirical winner for the planner’s specific case.

Our reason for choosing an ensemble over a single-strategy deployment is the no-free-lunch observation discussed in §5.1: no strategy dominates on every topology, the ranking is consistent enough to be useful but unstable enough that committing to one strategy in advance would sometimes be operationally wrong. The Köyceğiz table is an example — Min Damage Cut is tied for second there, and in a long-thin valley forest it could plausibly take first place. The planner has to see the comparison.

Two practical considerations reinforce the choice. The cost/performance trade-off between Lookahead (winner, 2.25 ms) and Betweenness Front (runner-up, 17.63 ms) is non-trivial; on graphs of $n > 300$ the relative costs invert, and a planner working on a large region needs Betweenness as a fallback because Lookahead’s top- $k = 6$ cap may no longer be enough. The explainability requirement (R5 in §2.3) also forces us to provide *why* a strategy is suggested: the recommendation engine generates a one-sentence rationale of the form *“For a hex-like topology with $k = 2$ and low front density, Lookahead’s pair-aware simulation usually beats single-vertex scoring because the second protected vertex changes which neighbours of the first matter next turn.”*

Limitations of the recommended solution. The Lookahead policy is hyperparameter-sensitive in one place (top- $k = 6$) and the recommendation engine’s classifier is rule-based, not learned. Both will need re-tuning if the operational regime changes (much larger graphs, much smaller k , stochastic spread).

Sensitivity. As in §5.1, the strategy gap is largest at $k = 2$; at $k = 3$ the gap already starts to compress, and we expect it to keep compressing at higher k because two well-chosen vertices already dominate, with diminishing returns from a third or fourth pick.

5.3 Further assessment of the recommended solution

Consistency with requirements and limitations. The ensemble respects every system requirement listed in §2.3: real geography input (R1), sub-second graph build via cache (R2), multiple comparable strategies (R3), per-turn animation (R4), recommendation card (R5), deterministic reproducibility (R6), offline operation after first call (R7). It honours every constraint listed in §2.4: ternary node state, fixed k per turn, deterministic spread, static topology, no settlement weighting at this stage.

Implementability. The full stack is implemented and demonstrated to work in the live suite at `firefighter-ie492.netlify.app/`. The total code base is about 6,000 lines (backend 3,500, frontend 2,500). The backend depends on widely available open-source packages (FastAPI, NetworkX, rasterio, SciPy). The frontend depends on Vite, React, TypeScript and Leaflet. No proprietary components are used. The Dockerfile and Fly.io configuration are committed; deployment to OGM’s internal infrastructure would consist of building the Docker image, pointing it at a tile cache directory of perhaps 5 GB for all Mediterranean tiles, and exposing the FastAPI port behind an internal authentication layer.

Sustainability. ESA WorldCover is updated annually; OpenStreetMap continuously. The strategies are static (no retraining required). The disk cache is small (under 500 MB after the Köyceğiz first

[10] OpenStreetMap contributors, “Planet OSM,” available at <https://planet.openstreetmap.org/>, accessed April 2026.

[11] European Commission Joint Research Centre, *Forest Fires in Europe, Middle East and North Africa 2023*, San-Miguel-Ayanz, J., Durrant, T., Boca, R. et al., Publications Office of the European Union, Luxembourg, 2024.

[12] M. Turco, J. von Hardenberg, A. AghaKouchak, M. C. Llasat, A. Provenzale and R. M. Trigo, “On the key role of droughts in the dynamics of summer fires in Mediterranean Europe,” *Scientific Reports*, vol. 7, art. 81, 2017, doi:10.1038/s41598-017-00116-9.

Appendix A — Detailed Test Configuration

Synthetic benchmark dimensions. * Graph sizes: $n \in \{30, 60, 100\}$. * Generator used in the paired benchmark: Delaunay planar. * Other generators implemented (long-thin Delaunay aspect 6:1, hexagonal lattice, Delaunay with 18% obstacle removal) are available in the engine for the live demo but were not included in the paired comparison reported in §5.1. * RNG seeds: 18 per size, fixed across strategies for paired comparison. * Starting vertices: 11 per graph instance, chosen to span low/medium/high-degree positions. * Budget: $k = 2$ (baseline). Sensitivity sweep also collected at $k = 1$ and $k = 3$. * Total: 4,752 single-strategy simulations (594 per strategy). * Random strategy seed: fixed at 0 for reproducibility.

Real-map case study. * Bbox: N 37.10, S 36.87, E 28.66, W 28.40 (Köyceğiz–Marmaris). * Area: 589 km². * Forest area extracted: 50,280 ha. * Vertex density rule: 500 ha per vertex → 101 vertices. * Edge candidates: 278 from Delaunay triangulation. * Active edges after the five-rule filter: 172. Blocked: 106. * Fire origin: vertex 0 (degree 7, the topological hub). * Budget: $k = 2$.

Appendix B — Strategy Implementation Pointers

For the reader who wants to inspect the code, the canonical file is `final suite/backend/firefighter_engine.py`. The class–method correspondence is:

Strategy	Class in firefighter_engine.py
Max Degree	<code>MaxDegree.select</code>
Max White	<code>MaxWhiteNeighbors.select</code>
Neighbours	
Min Cut Edge Front	<code>MinCutEdgeFront.select</code>
Min Damage Cut	<code>MinDamageCut.select</code>
Betweenness Front	<code>BetweennessFront.select</code>
One-Step Lookahead	<code>OneStepLookahead.select (top_k = 6, k_protect = 2)</code>
Hybrid Density Aware	<code>HybridDensityAware.select (threshold 0.18)</code>
Random	<code>RandomStrategy.select</code>